

Activity 02: System Utility Processes

Sharie Rhea

Southern New Hampshire University

CS 510: Operating Systems Principles and Applications

Brian Enochson

February 15, 2026

The provided Python file, `display_process_info.py`, retrieves the currently running processes on the system and displays some basic information about them. The code first retrieves the processes by utilizing the `process_iter` function in the `psutil` library. This function returns a `Process` iterator for all running processes on the machine, sorted by process ID or pid (Rodola, n.d.). The `attrs` argument takes a list of strings, which must be names for available process attributes (Rodola, n.d.). This data is then stored as a dictionary in the `info` attribute of each `Process` instance (Rodola, n.d.). If an empty list is provided for `attrs`, every attribute will be retrieved for each process, which is quite slow (Rodola, n.d.).

To run the program on your own device, please follow these steps. Note that `psutil` supports Linux, Windows, macOS, FreeBSD, OpenBSD, NetBSD, Sun Solaris, and AIX (Rodola, n.d.).

1. Install required dependencies.
 - a. Install Python v3.6+.
 - b. Install `psutil`: `pip install psutil`
2. Run the script in your preferred environment.
 - a. Integrated development environment (IDE): use the “run button” or conventional way to run a file in your preferred IDE.
 - b. Terminal: `python3 display_process_info.py`
3. Observe the formatted process info output in the terminal or your IDE’s terminal pane.

The script retrieves and displays four components for each process. The first piece of information is the process ID, or pid. This is a positive integer that uniquely identifies the process (Silberschatz et al., 2023). Note that if a process terminates, its pid may then be reused. As noted in the `psutil` documentation, it is not safe to assume that a pid will always refer to the

same process (Rodola, n.d.). The second component is the process name, which is the human-readable name for the process. This is often more useful to the user than the pid because it gives some indication as to what the process is and why it might be running. The third piece of info is the memory percentage. This is the percentage of available random-access memory (RAM) that the process is currently using. The final component is the central processing unit (CPU) percentage, which shows the utilization of the CPU. Note that this value is not split evenly between available CPU cores; rather it mimics the behavior of UNIX's `top` utility (Rodola, n.d.). The value can be greater than 100 percent if a process is running threads on multiple cores (Rodola, n.d.).

While developing this script did not pose a significant challenge, I did iterate through a few versions to improve the functionality. Initially, I utilized my IDE's autocompletion to retrieve each piece of info. I noticed there were attributes or functions for each piece of information I wanted to display: `process.pid`, `process.name()`, `process.memory_percent()`, and `process.cpu_percent()`. This worked, but after reading the `psutil` documentation more closely I noticed that the `process_iter` function stores the results of the defined `attrs` in a dictionary. I then modified the code to use this dictionary instead and noticed a minor speedup from ~0.060s to ~0.048s using the `bash time` utility. This makes sense intuitively, since calling the individual function for each component is likely repeating work as it needs to retrieve the process each time. The `process_iter` function is able to avoid this overhead by querying the four components together. However, the `psutil` library caches process info, so the incurred overhead is minimal.

References

- Rodola, G. (n.d.). *psutil documentation*. psutil.readthedocs.io. Retrieved February 11, 2026, from <https://psutil.readthedocs.io/en/latest/>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2023). *Operating System Concepts* (10th ed.). Wiley. <https://learn.zybooks.com/zybook/CS-510-10435.202617-1>